C# OOP Foundations & ADTs

Mastering Objects, Abstraction, and Data Structures in C#

Part 1: The Core OOP Pillars

Classes, Objects, Properties, and Methods

Class vs. Object

Class (The Blueprint)

A template or definition that describes what its instances will contain and do. It occupies no memory until instantiated.

Example: Architectural schematic for a building.

Object (The Instance)

A concrete entity created in system memory (Heap) using the class blueprint. It possesses its own unique state.

Example: The physical building constructed from the blueprint.

Properties vs. Methods

Properties (State / Nouns)

Characteristics or data fields belonging to an object. In C#, properties protect fields via safe accessors.

- ➔ `AccountBalance`
- ➔ `NodeValue`
- ➔ `IsFull`

Methods (Behavior / Verbs)

Functions defined inside a class that perform actions, execute operations, or alter the object's properties.

- ➔ `DepositMoney()`
- ➔ `InsertNode()`
- ➔ `ClearStack()`

Part 2: Data Architecture

Data Types, System Abstraction, and ADTs

C# Data Types: Memory Allocation

Value Types (Stack)

Stores data values directly. Copying allocates completely new, independent memory spaces.

- ➔ Primitives: `int`, `double`, `bool`
- ➔ Structures: `struct`

Reference Types (Heap)

Stores a memory address pointer pointing to heap memory. Copying duplicates only the address pointer.

- ➔ Classes: `class`
- ➔ Contracts: `interface`

Abstraction & Data Abstraction

Control Abstraction

Hiding complex systemic logic operations behind basic public interfaces.

Example: Pressing a button to brake a vehicle without knowing hydraulics mechanics.

Data Abstraction

Hiding how underlying data records are structured and managed internally inside memory storage devices.

Example: Accessing database records without managing physical arrays or files.

Abstract Data Types (ADTs)

An **ADT** is a theoretical design model specifying **what** data operations are available, but completely ignoring **how** they are coded.

The Conceptual ADT Model

- **ADT Name:** Stack
- **Behavior Rules:** Last-In, First-Out (LIFO)
- **Expected Methods:** Push(), Pop()

The C# Implementation Vehicle

Represented in C# using `interface` constructs:

```
public interface IStack {
    void Push(int item);
    int Pop();
}
```

Reviewing C# Classes: Implementation

Classes bring ADT interfaces to life by implementing concrete arrays, tracking variables, and operational logic.

```
public class ArrayStack : IStack
{
    private int[] _storage = new int[100]; // Hidden internal structure
    private int _topIndex = -1;

    public void Push(int item) => _storage[++_topIndex] = item;
    public int Pop() => _storage[_topIndex--];
}
```